



macOS privilege escalation

Vulnerability Disclosure

January 29, 2025

Prepared for:

Product Security

Apple

Prepared by: **Paweł Płatek**

Executive Summary

We found a user to root privilege escalation on the latest macOS (Sequoia 15.2) resulting from vulnerabilities in libinfo, mDNS Responder, and network_cmds projects. The privilege escalation is achieved by exploiting the traceroute6 SUID binary (installed by default at `/usr/sbin/traceroute6`) through a memory corruption in the libinfo library. Entry points for the memory corruption are `getipnodebyname`, `getaddrinfo`, and similar functions, implemented in the libinfo.

We verified the issue is exploitable on macOS Sequoia 15.2 (24C101) on the following machines:

- Macbook Pro M4 Max (16-core CPU, 128GB), kernel 24.2.0 (xnu-11215.61.5~2/RELEASE_ARM64_T6041) – on two different machines
- Macbook Air M1, 2020 (8-core CPU, 16GB), kernel 24.2.0 (xnu-11215.61.5~2/RELEASE_ARM64_T8103)

With the `/usr/sbin/traceroute6` binary sha1:

`cb2a317f0b81f703c6b964667afbf4b456c4390b`

We believe the issue can also be exploited on macOS with Intel CPUs. We haven't developed a fully working exploit there, but we attach a proof-of-concept (PoC) exploit that demonstrates RIP control. We verified this PoC on macOS Sequoia 15.2 (24C101) on the following machines:

- Macbook Pro Intel Core i9 (2,4GHz 8-core CPU, 16 logic cores, 32GB), kernel 24.2.0 (xnu-11215.61.5~2/RELEASE_X86_64) – on two different machines

This report consists of a few sections:

- List of the vulnerabilities
- Explanation of the call-path from `traceroute6` to the memory corruption
- Instructions for running the proof-of-concept exploits
- Details of the proof-of-concept exploit for Apple Silicon
- Details of the proof-of-concept for Intel x86
- A list of a few more, non-exploitable bugs

Vulnerabilities

1. In `mDNSResponder-2559.40.32` the `DNSSD_UDS_PATH` environment variable allows us to intercept communication with the mDNS Responder daemon by making a local

program use a Unix domain socket at a path of our choosing. This behavior allows us to target client-side DNS-related code (bypassing any mDNS Responder daemon's data validation).

```
char* uds_serverpath = getenv(MDNS_UDS_SERVERPATH_ENVVAR);
```

Figure 1: *mdnsResponder/mdnsShared/dnssd_clientstub.c#803*

The `getenv` is called in `dnssd_clientstub.c ConnectToServer` method. The bug can be verified by running the following command, which should fail to resolve the host: `DNSSD_UDS_PATH=/dev/null /usr/sbin/traceroute example.com`

We use this bug to reach the vulnerability #2 (below).

Ideally, privileged (SUID) programs should ignore the `DNSSD_UDS_PATH` environment variable and always use the default Unix domain socket (located at `/var/run/mdnsResponder`).

2. In [Libinfo-592](#), there is an integer overflow (leading to out-of-bound write on heap) in the `mdns_module.c _mdns_parse_domain_name` function.

```
1032 static char *
1033 _mdns_parse_domain_name(const uint8_t *data, uint32_t datalen)
1034 {
1035     int i = 0, j = 0;
1036     uint32_t len;
1037     uint32_t domainlen = 0;
1038     char *domain = NULL;
1039
1040     if ((data == NULL) || (datalen == 0)) return NULL;
1041
1042     /*
1043      * i: index into input data
1044      * j: index into output string
1045      */
1046     while (datalen-- > 0)
1047     {
1048         len = data[i++];
1049         domainlen += (len + 1);
1050         domain = reallocf(domain, domainlen);
1051
1052         if (domain == NULL) return NULL;
1053
1054         if (len == 0) break;          // DNS root (NUL)
1055
1056         if (j > 0)
1057         {
1058             domain[j++] = datalen ? '.' : '\0';
```

```

1059     }
1060
1061     while ((len-- > 0) && (0 != datalen--))
1062     {
1063         if (data[i] == '.')
1064         {
1065             /* special case: escape the '.' with a '\\' */
1066             domain = reallocf(domain, ++domainlen);
1067             if (domain == NULL) return NULL;
1068
1069             domain[j++] = '\\';
1070         }
1071
1072         domain[j++] = data[i++];
1073     }
1074 }
1075
1076 domain[j] = '\0';
1077
1078 return domain;
1079 }

```

Figure 2: *Libinfo/lookup.subproj/mdns_module.c#1032-1079*

The `_mdns_parse_domain_name` function is called with `data` and `datalen` arguments that were received from the intercepted Unix domain socket. The `datalen` is of `uint32_t` type, but is always called with `uint16_t` numbers (**`rdlen` variable in `_mdns_query_callback` function**). Normally, the first while loop (line 1046) would do at most 2^{16} iterations and buffer indexes (`int i` and `j` variables, line 1035) should never overflow.

However, the `datalen` may overflow in the second while loop (line 1061). The `datalen` is first compared to 0 and then decremented. If `datalen == 0`, then comparison fails, but decrementation happens anyway, turning `datalen` to `UINT32_MAX`. After the overflow the code jumps to the first while loop. Because `datalen` is now large, the loop continues iterating. Theoretically we can get out-of-bound read now, if the data's length is small. If the data's length is large, then the `i` and `j` indexes overflow to negative numbers after enough iterations. When that happens we have out-of-bound writes on the heap starting at `domain[INT_MIN]`, that is at `domain-231`.

To reach the overflow the second loop's first condition in line 1061 (`len-- > 0`) must evaluate to true. Since we control both the `len` (as we control the whole data buffer) and `datalen`, we can set `len` to be greater than `datalen` and pass the condition above. Note that the bug is reachable only locally, using the vulnerability

#1. The mDNS Responder daemon sanitizes data and datalen before sending them to the client, so the bug is not reachable remotely.

Simple PoC for the overflow:

```
int main() {
    uint64_t bufsize = (((uint64_t)2) << 30) - 2;
    char *buf = malloc(bufsize);
    if (!buf) {
        perror("malloc");
        return 1;
    }
    memset(buf, '.', 0x1000);
    memset(buf+0x1000, '\xff', bufsize-0x1000);

    memcpy(buf, "\x01\x44\x03\x45\x46\x47", 6);
    const uint8_t *rdata = (const uint8_t *)buf;
    uint16_t rdlen = 3;
    _mdns_parse_domain_name(rdata, rdlen);
}
```

Figure 3: simple proof-of-concept for vulnerability #2

To exploit the issue we need to layout the heap so that writes to `domain[INT_MIN]` are in a mapped memory. The attached proof-of-concept exploits set heap so that out-of-bound writes land in **libmalloc heap region's metadata**. Corrupting the metadata

allow us to write 8 bytes to (almost) arbitrary addresses using libmalloc's **recirc_list_extract** method.

Details of the exploit are provided later in this document.

The issue should be fixed by rewriting the `_mdns_parse_domain_name` function to be resilient against malicious inputs.

3. In **Libinfo-592** the `RES_DEBUG=1` environment variable causes a program that uses libinfo library to print stack and heap addresses. This bug allows us to bypass ASLR when exploiting SUID programs with vulnerability #2. The `getenv` function is called in **mdns_module.c _mdns_init** function.

```
if (getenv("RES_DEBUG") != NULL) _mdns_debug |= MDNS_DEBUG_STDOUT;
```

Figure 4: Libinfo/lookup.subproj/mdns_module.c#970

To test the issue run: `RES_DEBUG=1 /usr/sbin/traceroute6 ::1.`

Expected behavior for SUID programs would be to ignore the RES_DEBUG environment variable.

4. In `network_cmds-696`, there is an invalid revocation of privileges in the `traceroute6` program.

```
/* revoke privs */
seteuid(getuid());
setuid(getuid());
```

Figure 5: `network_cmds/traceroute6.tproj/traceroute6.c#631-633`

Call to `seteuid` followed by `setuid` results in a non-permanent privilege drop. Root can be restored by subsequent calls to `setuid(0)`. This behavior is documented in the `setuid`'s docstring.

Note that `traceroute` revokes privileges correctly. To fix the issue, the line with `seteuid` call should be removed. Moreover, the return value of the `setuid` call should be validated.

We could use this vulnerability in the exploit for macOS machines with Intel. The exploit for Apple Silicon does not need to use this bug, as it triggers memory corruption before privileges are dropped.

You can test the issue with:

```
// clang ./test.c -o ./test && sudo chown root ./test && sudo chmod 4755 ./test
#include <stdio.h>
#include <unistd.h>

int main(int argc, char const *argv[]) {
    printf("uid: %d\n", getuid());
    printf("euid: %d\n", geteuid());

    seteuid(getuid()); // comment this line to fix the issue
    setuid(getuid());

    printf("new uid: %d\n", getuid());
    printf("new euid: %d\n", geteuid());

    setuid(0); // sets effective UID=0
    setuid(0); // sets real UID=0

    printf("second new uid: %d\n", getuid());
    printf("second new euid: %d\n", geteuid());

    return 0;
```

```
}
```

Figure 6: simple proof-of-concept for vulnerability #4

Fixes for bugs #2 and #4 should be simple: minimal code modifications are needed. To fix bugs #1 and #3, libinfo and mDNS Responder libraries should reject respective environment variables if used from privileged programs. This could be accomplished by a similar method to what `libmalloc` does in the `set_flags_from_environment` function. Alternatively, the libraries should drop privileges when the environment variables are used, or the SUID programs should have an allowlist of accepted environment variables.

Memory corruption call-path

High-level execution flow is:

- `traceroute6` calls `libsystem_info.dylib` (libinfo)
- libinfo calls `libsystem_dnssd.dylib` (mDNS Responder client)
- mDNS Responder client connects to mDNS Responder daemon
 - Connection is intercepted with `DNSSD_UDS_PATH` envvar
 - mDNS Responder client connects to the provided Unix domain socket
- mDNS Responder client calls back to the libinfo
- libinfo calls the vulnerable `_mdns_parse_domain_name` function

Detailed execution flow is:

- `traceroute6` calls `getipnodebyname`
- Libinfo library handles the call, using `search_host_byname`
- The `search_host_byname` uses a few modules to resolve the host, including the `mdns_module` (called from `si_host_byname`). The `mdns_hostbyname` is called.
- The `_mdns_search_ex` is called (via `_mdns_search`); it calls `DNSServiceProcessResult` function
 - The `_mdns_search_ex` handles connections to the mDNS Response daemon (or the exploit) in a loop, reinitializing the connection up to 3 times.
 - The function implements timeouts
- The `DNSServiceProcessResult` is part of the mDNS Responder project. This function communicates with the exploit over a Unix domain socket. The function loops over DNS responses. For every response it:
 - Reads an IPC header
 - Calls `malloc` of our size (up to `UINT32_MAX`) and reads data to the allocated buffer
 - Calls `ProcessReply` which is the `ConnectionResponse` function

- The `ConnectionResponse` calls `op->ProcessReply` which is the `handle_query_response` function
- The `handle_query_response` calls `sdr->AppCallback` which is the `_mdns_query_callback`
- The `_mdns_query_callback` calls the vulnerable `_mdns_parse_domain_name` function

```
get_string(&data, end, name, kDNSServiceMaxDomainName);
rrtype = get_uint16(&data, end);
rrclass = get_uint16(&data, end);
rdlen = get_uint16(&data, end);
rdata = get_rdata(&data, end, rdlen);
ttl = get_uint32(&data, end);
```

Figure 7: `mDNSResponder/mDNSShared/dnssd_clientstub.c#1641-1646`

All data received over the Unix domain socket is stored in a malloc'ed buffer, and the `rdata` (`_mdns_query_callback` argument) points to this buffer (at some offset). The `rdlen` argument is parsed from the buffer; it must be smaller than the size of the allocated buffer (this is checked in the `get_rdata` method). The allocated buffer can have redundant data after `rdlen`, `rdata` and TTL.

Running proof-of-concept exploits

The proof-of-concept exploit for Apple Silicon is provided in `exp_poc.py` file. To execute the PoC a few additional files are needed (must be placed in the same directory as the python file):

1. `rvict1` or another `platform binary` without `Library Verification requirement` that can be executed outside of SIP-protected paths. The file can be copied from `/Library/Apple/usr/bin/rvict1` or downloaded. It must be placed outside of SIP-protected paths (placing it at the same location as `exp_poc.py` works). The exploit uses it to "bypass" Pointer Authentication Codes. We attach the `rvict1` file for completeness.
2. `do_pac.dylib` (arm64e binary). This library PAC-signs provided pointers. The exploit calls `rvict1` with `DYLD_INSERT_LIBRARIES=$(PWD)/do_pac.dylib` environment variable, so the library is executed with PAC's `JOP key (IA key)` configured and the same as SUID `traceroute6` would use.

3. `target_execve` (arm64 binary). The exploit makes `traceroute6` call `execvp(target_execve, NULL)`. This program prints some data to stdout, calls `setuid(0)` and opens a shell.

Running the exploit:

```
$ python3 ./exp_poc.py
```

The PoC should print some debugging information on the screen and open a root shell. If the shell is not opened (e.g., due to terminal issues), the end of a created `./stdout` file can be checked: it should have a "PWNERD" line followed by UIDs. We attach an example execution output (`example_output_exp_poc.txt` file). Execution time of the exploit is about 60 seconds, and about 4GB of RAM is used at the peak.

The PoC works reliably on both M4 and M1 machines, rarely failing because of insufficient heap spray. The few potential instabilities that may break the exploit can be compensated by changing some exploit's variables:

- The `max_cpus` variable, that is set to `hw.perflevel0.logicalcpu`. If the `traceroute6` is executed on `perflevel1` cores, then the `max_cpus` should be changed.
- There are a few timeouts in `libinfo` that may break the PoC on slower machines, under a heavy system load or because of specific `dyld` shared cache location. In these cases the heap spray may not spray enough regions. If that is the case then `FIXME` comments in the exploit's code point to the most likely problematic variables that can be tweaked. A more weaponized exploit could bypass the timeouts with the method used by exploit's `dns_response_empty_pages` function (described later).
- The `dyld` shared cache (`libsystem_kernel`'s `__DATA_DIRTY` section specifically) must not be located at an address containing `0x2e` byte (a dot character). This is because the exploit tries to overwrite a pointer in this section, and dots are escaped with slashes in the `_mdns_parse_domain_name` function. This problem is unlikely to occur but if it does then the system must be restarted for the exploit to work (PoC notifies about that problem). Note that a more weaponized exploit could circumvent this restriction by targeting a different pointer than one in the `libsystem_kernel`, for example.
- The offsets in `dyld` shared cache and PAC diversifiers may need adjustments (in `main` and `compute_where_what` functions). That should be only needed if new system libraries are compiled and pushed to macOS.

The exploit requires python3 to be installed, but it could be rewritten to a native executable. Execution of the files listed above may trigger the Gatekeeper's dialog boxes. Gatekeeper bypass is out of scope for this report, we assume the files can be executed. The exploit is unlikely to work from the Application Sandbox as the `/usr/sbin` location should not be reachable.

Exploit for Intel does not require any additional files and can be executed with:

```
$ python3 ./exp_poc_x86.py
```

Apple Silicon exploit explanation

PoC exploit flow is given below:

1. Required files are checked by the `check_files` method
2. `traceroute6` is started with proper environment variables (`RES_DEBUG`, `DNSSD_UDS_PATH`, `PATH`, `_STDBUF_0`)
 - a. `stdout` is redirected to a file, mostly for debugging purposes
 - b. `stderr` is redirected to a new PTY device; this is used later in the exploit
 - c. Argument flag `-g` is used so that `getipnodebyname` is called before calls to `seteuid/setuid`.
3. The filename of the file to be executed with `execvp` call is constructed dynamically based on the `traceroute6` process' PID, `pthread_self` address and `objects_in_use` constant. Exploit makes a symlink to the `target_execve` with the formatted filename.
4. Then the exploit creates a Unix domain socket (at `$(PWD)/my_socket`) and accepts connections. The `handle_one_connection` method implements simplified mDNS daemon protocol. It is called once for a single DNS request, and returns multiple DNS responses over the socket. The execution flow is now:
 - a. `dns_response_empty_pages`: this function allocates tiny, small, and medium regions for every libmalloc's magazine (~number of CPUs). This is needed to prevent allocation of new regions when the integer overflow is triggered in the `_mdns_parse_domain_name` function. Exploit knows if a new region was allocated with vulnerability #3 (leak of heap addresses). The function bypasses `_mdns_search_ex`'s timeouts by postponing sending last bytes of DNS responses so that the `loop in DNSServiceProcessResult`

continues without returning to `_mdns_search_ex` (the `morebytes` variable is set to true).

Moreover, this function puts payload (PAC-signed function pointers) on heap medium pages at a known address (again, vulnerability #3 gives us the address). Exploit puts the pointers at a known address to be able to sign them with PAC's context.

- b. `dns_response_spray_target`: it makes the `traceroute6` allocate a lot of small regions. Exploit sprays two types of objects: **PTR and SRV** DNS responses. The PTR objects are saved in the `context->host` object by the `_mdns_hostent_append_alias` function and are freed by the `_mdns_query_clear` method. The SRV objects are saved in `reply->srv` and are not freed by the `_mdns_query_clear` method.
- c. Connection is closed. This makes the `libinfo` (`_mdns_search_ex` function) reinitialize connection and **call `_mdns_query_clear` method**. Now all data allocated by `dns_response_empty_pages` and PTR objects allocated by the `dns_response_spray_target` are freed.
- d. `dns_response_overflow`: this function sends SRV DNS response that triggers integer overflow and out-of-bound write. It makes the `malloc` in `DNSServiceProcessResult` allocate `2^31-bytes_to_overflow` bytes, set `rdlen` to make `datalen` overflow quickly, send a dozen of dots so that `j` integer overflows before `i` (as dots are escaped), send data that will overwrite a small region's metadata, and finally send null-bytes to return from the `_mdns_parse_domain_name`. This method also closes `stderr` PTY (explained later).
- e. Now the connection should timeout, the `_mdns_query_clear` is called and the `_mdns_search` loop exits (with `-1` error).
- f. The **`_mdns_reply_clear`** is **called from `mdns_hostbyname`**. This method clears all the SRV responses (one from `dns_response_overflow` and many from `dns_response_spray_target`). One of the freed pointers is in the small region that has overwritten metadata. When this pointer is freed, the `libmalloc` writes an arbitrary pointer to our destination, and execution flow is intercepted.

Heap layout when out-of-bound writes happen is like this:

```

Malloc metadata
[some pages like Activity Tracking and MALLOC_LARGE metadata]
Dozens of tiny, small, and medium regions [initial regions are allocated here]
[empty space]
Stack
A few other small and medium regions
[__DATA of dyld shared cache]
[empty space]
A lot of small regions [sprayed pages, overwrite happens in one of these pages]
Large regions [data in _mdns_parse_domain_name, exactly 2^31-bytes_to_overflow bytes]
Large regions [domain in _mdns_parse_domain_name]

```

Figure 8: heap layout on Apple Silicon macOS

The size of large regions allocated for data are slightly less than 2^{31} . Writing to `domain[INT_MIN]` writes to one of the sprayed small regions just above the large regions.

Note that normally the heap grows “upwards” (towards lower addresses), because of how the `entropic_address` variable is used*. This can be easily observed on Intel x86 machines. However, on Apple Silicon the space between “Malloc metadata” and the initial heap addresses is small. After a dozen of regions are allocated there, subsequent regions are placed below the stack, and then below `__DATA` of dyld shared cache. From now on, the heap grows “downwards” (towards higher addresses). Because of that the large regions for the data buffer are allocated above the large regions for the domain buffer.

*More precisely the heap grows downwards from `entropic_address` address until a small `entropic_limit` is reached. Then `entropic_address` is decreased and new regions are allocated above previous ones, but again downwards from the updated `entropic_address` address. Effectively the heap grows upwards “in jumps”.

Note that location of dyld shared cache influences the required minimal number of small regions that must be sprayed for the exploit to succeed. The exploit assumes that at least 5 small pages are placed directly above the large region of the data variable (the `bytes_to_overflow` variable in PoC).

Exploit overwrites a small region’s metadata (`region_trailer` structure in `libmalloc`, `overflow_payload` in the exploit code) that is placed at the beginning of the region.

```

typedef struct region_trailer {
    struct region_trailer *prev;
    struct region_trailer *next;
    unsigned bytes_used;
    unsigned objects_in_use; // Used only by tiny allocator.
    mag_index_t mag_index;
}

```

```

volatile int32_t pinned_to_depot;
bool recirc_suitable;
// Locking: dispose_flags must be locked under the rack's region lock
rack_dispose_flags_t dispose_flags;
} region_trailer_t;

```

Figure 9: *libmalloc/src/magazine_zone.h#239-249*

```

typedef struct small_region {
    // This must be first (because SMALL_REGION_METADATA assumes it).
    region_trailer_t trailer;
    msize_t small_meta_words[NUM_SMALL_BLOCKS];
    oob_free_entry_s small_oob_free_entries[SMALL_OOB_COUNT];
    uint8_t pad[SMALL_REGION_PAD];
    region_cookie_t region_cookie;
    small_block_t blocks[NUM_SMALL_BLOCKS];
} * small_region_t;

```

Figure 10: *libmalloc/src/magazine_zone.h#464-472*

Most importantly, the region's prev and next pointers are set, but all fields in `region_trailer` and `small_meta_words` are overwritten too. Note that these two pointers are not protected with PAC nor with custom checksums. The `region_cookie` remains untouched.

When `_mdns_reply_clear` calls `free` with the pointer in the small region with corrupted metadata, we want the `libmalloc` to call the `recirc_list_extract` method in the `small_free_try_depot_unmap_no_lock` function with the following backtrace:

```

frame #5: 0x000000018ab1583c
libsystem_malloc.dylib`small_free_try_depot_unmap_no_lock + 180
frame #6: 0x000000018ab0c940 libsystem_malloc.dylib`free_small + 1332
frame #7: 0x000000018ad46fdc libsystem_info.dylib`_mdns_reply_clear + 64
frame #8: 0x000000018ad47b74 libsystem_info.dylib`mdns_hostbyname + 340
frame #9: 0x000000018ad47944 libsystem_info.dylib`search_host_byname + 168
frame #10: 0x000000018ad526f0 libsystem_info.dylib`gethostbyname2 + 184

```

Figure 11: *stack trace for recirc_list_extract*

More precise call-path inside `libmalloc` we want to target is:

- `free_small`
- `small_free_no_lock`
- `small_free_try_recirc_to_depot` (note that `CONFIG_RECIRC_DEPOT` must be configured, but it seems to be always set to 1)
- `small_free_try_depot_unmap_no_lock`
- `recirc_list_extract`

The `recirc_list_extract` gives us a write-what-where primitive, as we control `node->prev` and `node->next` pointers (figure 12). The primitive is limited by the requirement that both what and where addresses are writable. Before we describe what the exploit overwrites with this primitive, let's discuss requirements to reach the `recirc_list_extract`.

```
static MALLOC_INLINE void
recirc_list_extract(rack_t *rack, magazine_t *mag_ptr, region_trailer_t *node)
{
    // excise node from list
    if (NULL == node->prev) {
        mag_ptr->firstNode = node->next;
    } else {
        node->prev->next = node->next;
    }

    if (NULL == node->next) {
        mag_ptr->lastNode = node->prev;
    } else {
        node->next->prev = node->prev;
    }

    node->next = node->prev = NULL;
    mag_ptr->recirculation_entries--;
}
```

Figure 12: *libmalloc/src/magazine_inline.h#262-280*

The `small_meta_words` should be overwritten so that none has the `SMALL_IS_FREE` bit set. This prevents **forward/backward coalescing of chunks**. All `msizes` must be non-zero **to avoid errors**. Moreover, all `msizes` should be set to the same value (explained later, the exploit uses `0x3f` as the `msize`).

The `mag_index` must be set to `DEPOT_MAGAZINE_INDEX` (i.e., `-1`) to pass the **condition in the `small_free_try_recirc_to_depot`**. Exploit forces the region to be treated as “depotted” by setting this `mag_index`.

The **`bytes_used` and `pinned_to_depot` numbers must be zero**. The `pinned_to_depot` is just an integer we can set in the region's metadata. The `bytes_used` value is taken from the region's metadata **and decremented by the `original_size`** in the `small_free_no_lock` function. The `original_size` is `msize` taken from `small_meta_words`. Since the exploit has set all `small_meta_words` to the same `msize`, the `bytes_used` can be set to that value, so that decrementation results in exactly zero.

Finally, we must pass the condition `depot_ptr->recirculation_entries >= recirc_retained_regions`. The `recirc_retained_regions` is set to 2 by default. The `depot_ptr` is a pointer to a special libmalloc region that we have no direct control of (i.e., it is not the region we have overwritten). The `recirculation_entries` field is incremented in the `recirc_list_splice_last` and `recirc_list_splice_first` functions. We want to target the `recirc_list_splice_last` with this call-path:

1. `small_free_no_lock`
2. `small_free_try_recirc_to_depot`
3. `small_free_do_recirc_to_depot`
4. `recirc_list_splice_last`

```
if (small_magazine_below_recirc_threshold(small_mag_ptr)) {  
    return small_free_do_recirc_to_depot(rack, small_mag_ptr, mag_index);  
}
```

Figure 13: `libmalloc/src/magazine_small.c#1270-1271`

For this call-path to be executed we need the `small_magazine_below_recirc_threshold` function return true when freeing some pointer. Conditions in this function means that we need to have at least two regions in a magazine that have held a lot of data that was already freed. We could just allocate a lot of regions and then free them all for the condition to pass (during one of the frees), but that would cause the regions to be **completely unmapped from memory** (during the last free). Instead, we need to allocate a lot of regions and then free most of the data inside them, but not all data. This is exactly what the exploit does in the `dns_response_spray_target` function in the exploit: allocates a lot of PTR and SRV objects, and then free only PTRs. This makes the `recirc_list_splice_last` method to be called multiple times. When the free in the corrupted region occurs, the `depot_ptr->recirculation_entries` should be much greater than 2.

Now the code reaches the `recirc_list_extract`. Exploit sets:

- prev pointer ("write what") to an address in a medium region where our payload (function pointers) was written
- next pointer ("write where") to an address of the `libkernel_funtions pointer`.

```
__attribute__((visibility("hidden")))  
_libkernel_functions_t _libkernel_functions;
```

Figure 14: `xnu/libsyscall/wrappers/_libkernel_init.c#80-81`

The `recirc_list_extract` function overwrites `libkernel_funtions` pointer with the heap address, so now we control **all `libkernel_funtions`' function pointers**.

```
typedef const struct _libkernel_functions {
    /* The following functions are included in version 1 of this structure */
    unsigned long version;
    void* (*LIBKERNEL_FUNCTION_PTRAUTH(dlsym))(void*, const char*);
    void* (*LIBKERNEL_FUNCTION_PTRAUTH(malloc))(size_t);
    void (*LIBKERNEL_FUNCTION_PTRAUTH(free))(void*);
    void* (*LIBKERNEL_FUNCTION_PTRAUTH(realloc))(void*, size_t);
    void (*LIBKERNEL_FUNCTION_PTRAUTH(pthread_exit_if_canceled))(int);
}
```

Figure 15: `xnu/libsyscall/wrappers/_libkernel_init.h#61-68`

After `recirc_list_extract`, the `small_free_detach_region` function is called (figure 16). It returns the number of allocated objects in the region, using `small_meta_words` metadata for computation. Since we have overwritten the `small_meta_words`, the returned value (`objects_in_use` variable) is deterministic and always 259. Because the value is not zero, the `malloc_zone_error` function is called.

```
1134     static region_t
1135     small_free_try_depot_unmap_no_lock(rack_t *rack, magazine_t *depot_ptr,
region_trailer_t *node)
1136     {
1137         if (0 < node->bytes_used || 0 < node->pinned_to_depot ||
depot_ptr->recirculation_entries < recirc_retained_regions) {
1138             return NULL;
1139         }
1140
1141         // disconnect first node from Depot
1142         recirc_list_extract(rack, depot_ptr, node);
1143
1144         // Iterate the region pulling its free entries off the (locked) Depot's
free list
1145         region_t sparse_region = SMALL_REGION_FOR_PTR(node);
1146         int objects_in_use = small_free_detach_region(rack, depot_ptr,
sparse_region);
1147
1148         if (0 == objects_in_use) {
1149             if (!rack_region_remove(rack, sparse_region, node)) {
1150                 return NULL;
1151             }
1152             depot_ptr->num_bytes_in_magazine -= SMALL_HEAP_SIZE;
1153
1154             // Caller will transfer ownership of the region back to the OS
with no locks held
1155             MAGMALLOC_DEALLOCREGION(SMALL_SZONE_FROM_RACK(rack), (void
*)sparse_region, (int)SMALL_REGION_SIZE); // DTrace USDT Probe
1156             return sparse_region;
}
```

```

1157
1158     } else {
1159         malloc_zone_error(rack->debug_flags, true,
1160             "small_free_try_depot_unmap_no_lock objects_in_use
not zero: %d\n", objects_in_use);
1161         return NULL;
1162     }
1163 }

```

Figure 16: *libmalloc/src/magazine_small.c#1134-1163*

The `malloc_zone_error` calls `malloc_vreport` which constructs an error message, conditionally calls `_simple_put`, and calls `abort`. We want the code to call the `_simple_put`, so we need the `WRITE_TO_DEBUG_FILE` macro to be true.

```

if (WRITE_TO_DEBUG_FILE(flags)) {
    _simple_put(b, malloc_debug_file);
}

```

Figure 17: *libmalloc/src/malloc_printf.c#223-224*

```

#define WRITE_TO_DEBUG_FILE(flags) \
    (((flags) & MALLOC_REPORT_NOWRITE) == 0 && \
     ((debug_mode == DEBUG_WRITE_ALWAYS) || \
      (debug_mode == DEBUG_WRITE_ON_CRASH && ((flags) & \
      MALLOC_REPORT_CRASH))))

```

Figure 18: *libmalloc/src/malloc_printf.c#58-61*

The `MALLOC_REPORT_NOWRITE` is not set, and `debug_mode` is set to `DEBUG_WRITE_ALWAYS` in `malloc_print_configure` function if `stderr` is a TTY device. Note that `malloc_print_configure` is called only if no `Malloc*` and `_Mallo*` environment variables are set.

```

// Default is to write to stderr only if it's a tty.
if (isatty(STDERR_FILENO)) {
    debug_mode = DEBUG_WRITE_ALWAYS;
}

```

Figure 19: *libmalloc/src/malloc_printf.c#85-88*

When the `WRITE_TO_DEBUG_FILE` returns true the `_simple_put` from the `libplatform` library is called. This function calls `_flush` and `_flush` executes `write` syscall. If the syscall fails (returns an error) then the `error` function is called. The `error` calls `_pthread_exit_if_canceled` which calls `_libkernel_functions->pthread_exit_if_canceled` that we control.

```

cerror(int err)
{
    _pthread_exit_if_canceled(err);
    return cerror_nocancel(err);
}

```

Figure 20: *xnu/libsyscall/custom/errno.c#75-79*

```

_pthread_exit_if_canceled(int error)
{
    return _libkernel_functions->_pthread_exit_if_canceled(error);
}

```

Figure 21: *xnu/libsyscall/wrappers/_libc_funcptr.c#124-127*

To force write to error out the exploit closes the stderr PTY before the `malloc_zone_error` is called. The full stacktrace when the IP register is taken over is given below:

```

frame #0: 0x000000018acc07b4 libsystem_kernel.dylib`cerror + 44
frame #1: 0x000000018acc2a28 libsystem_kernel.dylib`write + 28
frame #2: 0x000000018ad3855c libsystem_platform.dylib`_flush + 56
frame #3: 0x000000018ab15d20 libsystem_malloc.dylib`malloc_vreport + 512
frame #4: 0x000000018ab3e498 libsystem_malloc.dylib`malloc_zone_error + 100
frame #5: 0x000000018ab1583c
libsystem_malloc.dylib`small_free_try_depot_unmap_no_lock + 180
frame #6: 0x000000018ab0c940 libsystem_malloc.dylib`free_small + 1332
frame #7: 0x000000018ad46fdc libsystem_info.dylib`mdns_reply_clear + 64
frame #8: 0x000000018ad47b74 libsystem_info.dylib`mdns_hostbyname + 340
frame #9: 0x000000018ad47944 libsystem_info.dylib`search_host_byname + 168
frame #10: 0x000000018ad526f0 libsystem_info.dylib`gethostbyname2 + 184

```

Figure 22: *stack trace for cerror*

From here the exploit does JOP chain using functions in the `libkernel_functions`. JOP is implemented in the `compute_where_what` method in the PoC code. The flow is:

1. `_libkernel_functions->_pthread_exit_if_canceled` calls `libsystem_kernel:reallocf`
 - a. Sets `x21` to address of `libkernel_functions`
 - b. Calls `_libkernel_functions->realloc`
2. `_libkernel_functions->realloc` calls `_posix_spawnattr_destroy`
 - a. Sets `x0` to `x20` where `x20` is a pointer to the formatted `malloc_zone_error` error message
 - b. Calls `_libkernel_functions->free`
3. `_libkernel_functions->free` calls `execvp`

- a. `x0` is a pointer to "traceroute6(PID,SELF) malloc:
small_free_try_depot_unmap_no_lock objects_in_use not: 259\n" string (the error message)
- b. `x1` is zero

The `execvp` sets the `environ` for us and looks for the executable in `PATH` environment variable. The exploit had set the `PATH` to current working directory and had created a symlink to the `target_execve` executable so that `execvp` can find it. Now the `target_execve` calls `setuid(0)` to set real UID from effective UID, and spawns a shell.

The exploit indicates a need to improve security controls of `libmalloc`. While most of `libmalloc`'s pointers are protected with custom checksums, the `prev` and `next` pointers in regions metadata are not. We suggest adding similar checksums to these pointers or even better to sign them with PAC.

Moreover, the exploit demonstrates limitations of Pointer Authentication Codes when it comes to protecting programs against local exploits. Because JOP (IA) PAC keys are shared between all platform binaries, any platform binary can be targeted to get PAC-signing ability. This is a well known issue. As a partial remediation for the problem we suggest that root SUID programs should get a unique JOP (IA) PAC key, distinct from not-SUID platform binaries. Or more broadly, platform binaries running with root privileges should get different JOP PAC keys than platform binaries executed with lower privileges.

Intel exploit explanation

The proof-of-concept exploit for Intel x86 (`exp_poc_x86.py` file) only demonstrates RIP control. It should crash on `0xaaaaaa` address (`rip_target` variable in the python code). Constructing a JOP/ROP chain from here should be easy (no PAC), but we didn't want to postpone the reporting, as the Apple Silicon PoC already demonstrates exploitability.

The main structure of the exploit is the same as for Apple Silicon. The differences are explained below.

Exploit targets calls to `getaddrinfo` and `gethostbyname` (called from `as_setup`) instead of `getipnodebyname` in `traceroute6`. The PoC needs two calls into `libinfo`, because heap layout/behavior is different on Intel than on Apple Silicon. Because both function calls are executed after root privileges are revoked, the JOP/ROP chain would need to call

setuid(0) followed by execve (instead of just execve as in the Apple Silicon exploit). The privileges can be restored because of vulnerability #4.

Heap layout when traceroute6 crashes is like this:

```
Malloc metadata  
[empty space]  
Malloc nano region  
[empty space]  
A lot of small regions [sprayed pages, overwrite happens in one of these pages]  
Large regions [data in _mdns_parse_domain_name, exactly 2^31-bytes_to_overflow bytes]  
Large regions [domain in _mdns_parse_domain_name]  
Dozens of tiny, small, and medium regions [initial regions are allocated here]  
[empty space]  
Stack
```

Figure 23: heap layout on Intel macOS

The space between stack and Malloc nano region is large enough to hold all heap regions. On Apple Silicon the corresponding space is much smaller and the heap grows below the stack.

To make the out-of-bounds write land in sprayed small regions the heap must grow downwards (towards higher addresses). The exploit forces this behaviour by allocating ~8GB of data on the heap and then freeing it all. This **decreases the entropic_address variable by a large number**, creating ~8GB of empty space where new regions are allocated downwards.

Exploit implements this part in the dns_response_big_alloc function. The function uses CNAME DNS response to force the ~8GB allocations (from ~2GB of data sent). All this allocated data is freed by timing out the getaddrinfo call. To make the getaddrinfo not return error and traceroute6 to continue execution to the gethostbyname call, the hostname used by the exploit (hostname variable, "localhost" by default) must have an IPv6 address in the /etc/hosts file.

About 4GB of data is written to stdout by traceroute6 because of the ~8GB allocations. Exploit handles that by redirecting traceroute6's stdout to a file and doing seeks on the file (the stdout_size method of MyProcess class in the exploit). This avoids reading all 4GB line-by-line.

Pointer Authentication (PAC) is not available on Intel so the exploit does not have to sign pointers. Therefore it puts the payload (rip_target function pointer) in environment variables, instead of the heap. The exploit learns the address of the environment variables with vulnerability #3.

The exploit can get RIP control by spraying tiny regions (instead of small) and **overwriting `lite_zone` pointer in `libmalloc`** (instead of `libkernel` functions). To test this other exploitation path the `spray_type` variable in the PoC should be changed. Using tiny regions and `lite_zone` overwrite is less stable than the default exploitation path.

Additional, not exploitable bugs

1. Invalid search order in `Libinfo/search_module.c si_module_static_search`. Docstring says that flat file is consulted before mDNS, but the code is vice-versa. Not sure if the doc or the code is invalid. We use the fact that the file is consulted after mDNS in the Intel PoC.

2. Vim is not listed in `xnu/mach_loader.c process_is_plugin_host`. Vim can load plugins (dylibs) via `libcall command`. Because vim is not listed in the `hardening_exceptions` array, this functionality does not work.

3. memmove buffer overflow in `traceroute inetname`.
The `domain` buffer has size `MAXHOSTNAMELEN+1`.
If the `gethostname` function returns `name` without dots, then `gethostbyname(domain)` is called, and the result is memmoved to the `domain` buffer. The `gethostbyname` may return a longer string than `MAXHOSTNAMELEN+1`, and so `memmove` may write out-of-bounds.

The bug is not exploitable because of FORTIFY_SOURCE.

The bug can be tested by setting no-dot hostname and changing /etc/hosts:

[illegible]

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review projects, supporting client organizations in the technology, defense, and finance industries and government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, and Zoom.

Trail of Bits also operates a center of excellence with regard to blockchain security. Notable projects include audits of Algorand, Bitcoin SV, Chainlink, Compound, Ethereum 2.0, MakerDAO, Matic, Uniswap, Web3, and Zcash.

To keep up to date with our latest news and announcements, please follow [@trailofbits](#) on Twitter and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2025 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.